

FEUILLE DE ROUTE DE FORMATION

Devenir AI Engineer

La carte de compétences complète : skills, outils, socle d'ingénierie et workflow

FORMULE À RETENIR

AI Engineer = Ingénieur logiciel + maîtrise des LLM



Skills



Works With



Tools



Core Skills



Workflow



Qu'est-ce qu'un AI Engineer ?

Pas (forcément) un chercheur qui entraîne des modèles géants depuis zéro : un ingénieur logiciel qui construit des applications fiables autour de LLM — assistants, moteurs de recherche documentaire, agents automatisés.

À côté des compétences « IA » (prompts, RAG, fine-tuning), on trouve un socle d'ingénierie classique : Python, Docker, Git, SQL, FastAPI, cloud...

LES 5 BLOCS DE LA CARTE DE COMPÉTENCES



Skills

Le cœur du métier : prompt, contexte, RAG, fine-tuning, garde-fous



Works With

Données non structurées, API LLM, bases vectorielles



Tools

LangChain, LangGraph, n8n, Hugging Face, Supabase...



Core Skills

Python, Docker, Git, SQL, FastAPI, cloud



Workflow

Le cycle de vie itératif d'un projet IA en production

Ce cours transforme cette carte en un programme d'apprentissage concret.



Module 1 — Compétences fondamentales (1/2)



1. Prompt Engineering

Formuler les instructions pour des réponses fiables : few-shot, chain-of-thought, sorties structurées, rôles et contraintes.

Base que DSPy vient automatiser.



2. Context Engineering

Décider quelle information entre dans la fenêtre de contexte : prompt système, RAG, mémoire, outils, schémas de sortie.

Trop d'info tue l'info.



3. Building With LLMs

Intégrer un LLM en prod : appels API, function calling / tool use, sorties structurées, cache, gestion d'erreurs.

Le modèle appelle des fonctions.



4. LLM Fine-Tuning

Adapter un modèle à son domaine en modifiant ses poids — LoRA / PEFT. Seulement quand prompting + RAG ne suffisent pas.

Lié à distillation & GRPO.



Module 1 — Compétences fondamentales (2/2)



5. RAG Design

Concevoir un pipeline RAG : ingestion (chunking, embedding, index), retrieval hybride + reranking, génération. Variantes : RAG multimodal, agentic RAG, Chain-of-RAG.

Voir le cours précédent.



6. Guardrails

Protections contre les dérapages : validation des entrées (injection, sujets interdits) et sorties (format, PII, toxicité, hallucinations).

Sans garde-fous, pas de prod responsable.



7. Embeddings & Search

Un embedding encode le sens d'un texte/image. Similarité cosinus pour retrouver du sens sans partager les mots. + recherche hybride, filtres, rerankers.

Le cœur de la recherche sémantique.



Ce bloc — Skills — est le cœur du métier : maîtriser la chaîne complète prompt → contexte → RAG → fine-tuning → garde-fous.



Module 2 — Les matériaux de travail

Ce que l'AI Engineer manipule au quotidien pour construire ses systèmes — et le défi associé (encadré).



Unstructured Data Données non structurées

PDF, emails, images, audio, pages web

Parser, nettoyer, découper (chunking) proprement — 80 % du travail réel !



LLM APIs

OpenAI, Anthropic, etc.

Gérer coûts, latence, limites, versions de modèles, fallbacks



Vector Databases Bases vectorielles

pgvector, Chroma, Pinecone, Supabase...

Stocker, indexer et interroger des millions d'embeddings rapidement



Module 3 — La boîte à outils



LangChain

Briques pour enchaîner appels LLM, outils et mémoire (chains).



LangGraph

Agents en graphe (nœuds, états, boucles) — flux contrôlables.



n8n

Automatisation low-code : emails, CRM, webhooks sans tout coder.



Hugging Face

Le hub des modèles, datasets, Transformers/PEFT — fine-tuning.



Supabase

Backend Postgres + pgvector, auth, stockage — héberger l'app IA.



Guardrails

Frameworks de validation des sorties — le concept du Module 1, outillé.



OpenAI / Anthropic APIs

Les moteurs LLM eux-mêmes.



Module 4 — Le socle d'ingénierie

💡 *Un modèle brillant dans une application cassée = un produit cassé. Ce socle fait la différence entre une démo et un produit.*



Python

Le langage principal de l'écosystème IA



Docker

Emballer l'app pour qu'elle tourne partout à l'identique



Git

Versionner le code... et les prompts



SQL

Interroger les données, y compris via pgvector



FastAPI

Exposer son RAG/agent en API rapide et documentée



Deployment

Mettre en production : CI/CD, scaling, environnements



Cloud Platforms

AWS / GCP / Azure — héberger, surveiller, facturer



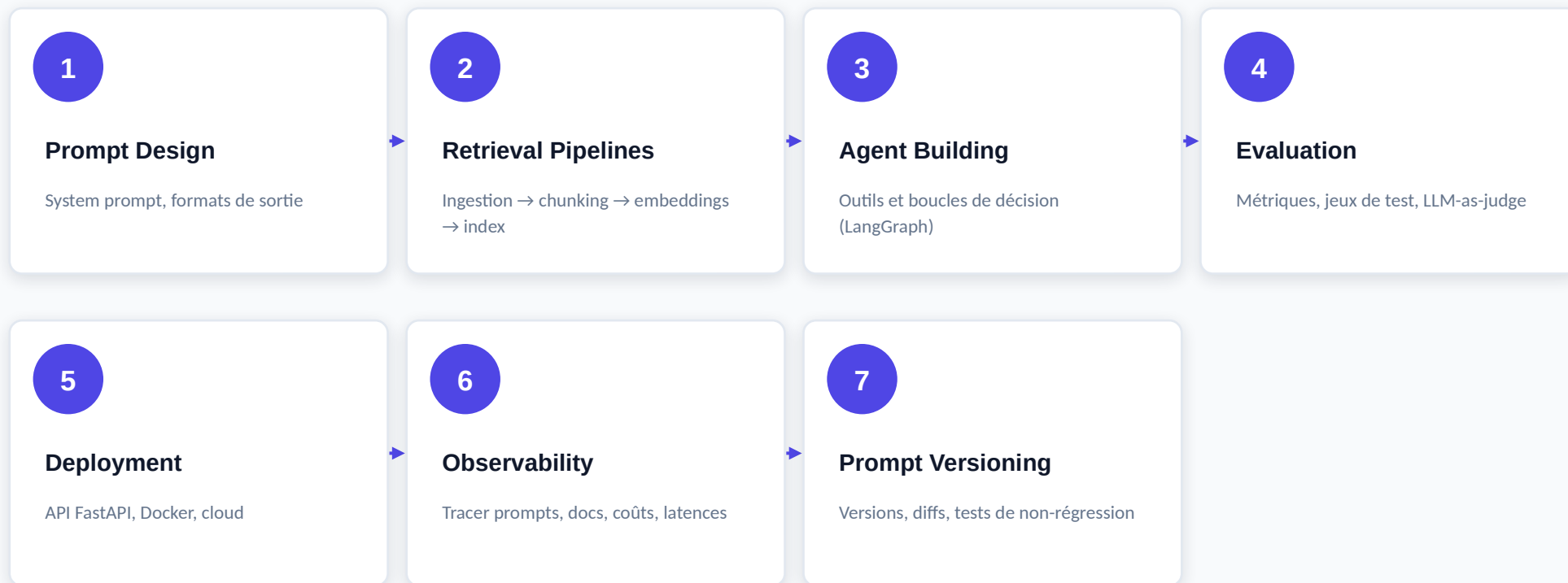
REST APIs

Le langage universel entre les services de l'app



Module 5 — Le workflow réel d'un projet

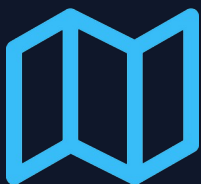
L'ordre chronologique d'un projet type — et le plan de ton futur portfolio. Le cycle est itératif.



retour à
l'étape 1



L'observabilité nourrit l'évaluation, qui nourrit le prompt design... Sans évaluation, chaque changement est un pari.



Module 6 — Ta feuille de route

Étape	Objectif	Compétences mobilisées
1	Socle logiciel	Python, Git, SQL, FastAPI, Docker
2	Premiers pas LLM	APIs OpenAI/Anthropic, prompt engineering
3	Recherche sémantique	Embeddings, base vectorielle (Supabase/pgvector)
4	Projet fil rouge : assistant Q&A sur PDF	RAG design, LangChain
5	Fiabiliser	Guardrails, évaluation, context engineering
6	Automatiser	Agents LangGraph, n8n
7	Industrialiser	Deployment, cloud, observability, versioning
8	Spécialiser	Fine-tuning (Hugging Face, LoRA) si besoin

PROJET FIL ROUGE RECOMMANDÉ

Un assistant qui répond aux questions sur une collection de PDF — API FastAPI, base vectorielle Supabase, guardrails, évaluation sur 20 questions, déploiement Docker + cloud, traçage des requêtes. Il couvre les 5 blocs de la carte.

CONCLUSION

L'AI Engineer assemble données, embeddings, LLM et garde-fous en systèmes fiables et observables.

Bloc	Message clé
Skills	Maîtriser la chaîne : prompt → contexte → RAG → fine-tuning → garde-fous
Works With	Vivre au milieu des données non structurées, API LLM et bases vectorielles
Tools	LangChain/LangGraph pour orchestrer, Hugging Face pour les modèles, Supabase pour les données
Core Skills	Rester un ingénieur logiciel solide : Python, Docker, Git, SQL, FastAPI, cloud
Workflow	Concevoir → récupérer → agentifier → évaluer → déployer → observer → versionner

EN UNE PHRASE

L'AI Engineer traite ses prompts comme du code.